CITY UNIVERSITY
MALAYSIA

FACULTY OF INFORMATION TECHNOLOGY

OBJECT-ORIENTED PROGRAMMING FUNDAMENTALS IN JAVA

SELF-REFLECTIVE REPORT

ASSIGNMENT 1 - INDIVIDUAL (20%)

PREPARED BY:

No.	Name	Student ID	Program	Passport Number	Class Code
1	ALI MOHAMMED RIYADH YASEEN FAREA	202501010286	BCSSE	14036858	BIT1123

Lecturer: Sir Nazmirul Izzad Bin Nassir

Due Date: 21 August 2026

Table of Contents

1. Introduction

This report reflects on my learning journey throughout the Object Oriented Programming tutorials from Week 1 to Week 10. I am currently at the beginning of my second year in the Bachelor of Computer Science (Software Engineering) programme. Before completing these tutorials, I understood some basic programming ideas, but I had limited experience in organising a Java program into several classes. The weekly activities helped me move from simple output and conditional statements to classes, inheritance, abstraction, file handling, and a graphical user interface. They also gave me practical experience in keeping my work in one GitHub repository. In this report, I explain what I learned from each tutorial, the main difficulties I faced, how I approached them, and the skills I plan to develop next.

2. Knowledge Gained from Each Tutorial

2.1 Week 1 - Java Basics

In Week 1, I became familiar with the GitHub and Codespaces workflow and created the first tutorial folder. The HelloWorld program showed me the basic structure of a Java class and the main method. In StudentGrade, I used variables to store a student name and mark, and I used if-else conditions to choose a grade. This was a simple exercise, but it helped me understand that the order of conditions is important. For example, the program checks the highest grade boundary first before moving to the lower boundaries. I also learned that Java is case-sensitive and that the public class name must match the file name.

2.2 Week 2 - Classes and Objects

Week 2 introduced me to classes and objects through the Student example. I created attributes for the student's name, age, and GPA, then used a constructor to provide the starting values. In Main.java, I created a Student object and called displayInfo, study, and takeExam. This helped me see the difference between a class, which is the design, and an object, which is a usable instance of that class. I also understood why methods are useful: each method has one clear responsibility, so the main method does not need to contain every line of the program.

2.3 Week 3 - Inheritance

In Week 3, I learned how inheritance can reduce repeated code. Person stores the common name and ID, while Student and Lecturer extend Person. The subclasses call the parent

constructor using super(name, id), so the same attributes do not need to be declared again. This activity helped me understand the meaning of an 'is-a' relationship: a Student is a Person and a Lecturer is also a Person. I also used getName and getId to access the private information in a controlled way. The example made inheritance more practical than a definition from a lecture slide.

2.4 Week 4 - Method Overriding and Polymorphism

Week 4 continued the same scenario and focused on method overriding. Person, Student, and Lecturer each have an introduce method, but the student and lecturer versions produce different messages. The @Override annotation helped me confirm that the method matches the one in the parent class. The most useful part was creating a Student object through a Person reference. When introduce was called, Java used the Student implementation. This showed me that polymorphism is not only about different class names; it allows one parent type to represent different child objects while keeping their own behaviour.

```
Person p2 = new Student("Ali", "2000");
p2.introduce();
```

2.5 Week 5 - Encapsulation

In Week 5, I applied encapsulation in a student information system. The student ID, name, CGPA, and programme are private, so another class cannot change them directly. I added getters to read the values and setters to update them. At first, getters and setters looked like extra code, but the tutorial showed their purpose. They create a controlled path to the data, and validation can be added later without changing the code that uses the Student class. The documentation questions also made me explain why public attributes would make the data less protected and harder to control.

2.6 Week 6 - Extending an Employee Class

Week 6 gave me more practice with inheritance using Employee and Lecturer. Employee contains the shared ID and name, while Lecturer adds a subject and department. The Lecturer constructor passes the common values to Employee with super and stores its own values separately. I learned that a subclass should add information or behaviour that is specific to it instead of copying all the parent code. Calling displayInfo and displaySubject also showed how methods from both levels of the class relationship can be used on the same object.

2.7 Week 7 - Abstraction

In Week 7, I used an abstract Appliance class for a smart home scenario. Every appliance has a brand and can turn on or off, so these behaviours are implemented once in the parent class. However, the operate method is abstract because each appliance works differently. WashingMachine prints a washing action, while Refrigerator prints that it stores food and beverages. This helped me understand that abstraction provides a common structure but leaves the specific operation to the child class. It also showed why an abstract Appliance is not created directly; the program creates a concrete appliance that completes the missing behaviour.

```
public abstract void operate();
```

2.8 Week 8 - ArrayList and User Input

Week 8 moved from fixed object examples to a small to-do list. I created an ArrayList<String> to store tasks and used Scanner to read three entries from the user. A for loop collects the tasks, and another loop displays them with numbers. This tutorial improved my understanding of a collection that can grow while the program runs. It was also useful to see how a loop, user input, and a collection work together. Instead of creating a separate variable for every task, the ArrayList keeps the related data in one structure.

2.9 Week 9 - File Handling

Week 9 extended the to-do list by saving the tasks in task.txt. I used BufferedWriter and FileWriter to write each item on a new line. I then used BufferedReader and FileReader to load the saved tasks. The try-with-resources structure was important because it closes the file automatically. I also learned that file operations may fail, so IOException must be handled instead of assuming that the file is always available. Seeing the tasks appear again after they were written helped me understand the difference between temporary data in memory and data that remains in a file.

2.10 Week 10 - Java Swing and Event Handling

In Week 10, I completed a small Programming Quiz Battle using Java Swing. Questions stores the question, two options, and the correct answer. QuizBattleGUI creates the frame, labels, and buttons. The buttons register an ActionListener, and actionPerformed checks which button was clicked. I learned that a graphical program does not follow only a top-to-bottom sequence; it waits for an event from the user. Separating the question data from the GUI also showed me a simple way to keep the program organised. This was a useful final exercise because it combined objects, methods, and user interaction.

3. Challenges Encountered

The main challenge for me was understanding inheritance and abstraction. At first, I could read the individual classes, but I found it harder to follow which class owned a variable or method when several classes were connected. The use of super in a child constructor was also confusing because the values were passed to code in a different file. Method overriding added another question: I needed to understand why a Person reference could call the Student version of introduce. In Week 7, I also needed time to understand why Appliance could contain normal methods and an abstract method at the same time.

File handling and Swing required extra attention for different reasons. With file handling, the program depends on a file path and must deal with IOException. With Swing, I needed to follow how an ActionEvent moves from a button to the actionPerformed method. These parts were new to me, but I did not treat them as separate large systems. I focused on the small task required by each tutorial and checked the output before adding the next part.

4. How the Challenges Were Overcome

I overcame the inheritance and abstraction difficulties by testing the code in small steps. For the Person example, I first checked the parent constructor and introduce method. I then added Student, compiled the files, and compared the message with the expected output before adding Lecturer. This made it easier to see that super sets the inherited information, while @Override changes the behaviour. I followed a similar process with Appliance: I checked the shared brand and power methods first, then implemented operate separately in WashingMachine and Refrigerator.

When an error appeared, I tried to identify its cause instead of changing many lines at once. I checked class names, file names, constructor parameters, and method signatures. I also used the expected output as a clear target. For the file exercise, I confirmed that tasks were written before trying to read them. For the GUI, I checked the question and button text before checking the result label. This gradual approach helped me understand the program and made debugging less confusing.

5. Improvements in Java Programming Skills

Across the tutorials, my Java skills improved in both coding and organisation. I am more comfortable writing classes, constructors, and methods, and I can separate a program into files

with different responsibilities. I also improved at reading compiler messages. Problems such as a class name not matching its file or an incorrect constructor argument now give me a clearer place to start checking. The weekly progression helped me move away from putting all logic in the main method.

I also gained experience outside the Java syntax itself. I organised the work into consistent tutorial folders, kept the source files in one GitHub repository, and used README documentation to explain the projects. ArrayList, file handling, and Swing showed me that Java can be used for more than short console examples. I still need more practice, but I can now understand how different classes and Java libraries work together in a small application.

6. Understanding of Object-Oriented Programming Concepts

My understanding of classes and objects became clearer in Week 2. The Student class defines the data and actions, while s1 is an object with its own name, age, and GPA. Encapsulation in Week 5 added another level to this understanding. Making the fields private means the class controls how its data is accessed, and getters and setters provide the public connection to that data.

Inheritance became clear when I compared Person with Student and Lecturer. The parent contains what is common, and each child adds or changes what it needs. Polymorphism was demonstrated when a Person reference held a Student or Lecturer object and the correct introduce method ran. This makes the program more flexible because code can work with a common type without losing the behaviour of the actual object.

Abstraction was demonstrated by Appliance. The parent class defines what all appliances can do, such as displaying a brand and turning on or off. The abstract operate method requires each concrete appliance to provide its own action. From this example, I understood that abstraction hides unnecessary implementation details and gives related classes a common rule to follow. Together, these concepts make code easier to reuse, extend, and understand.

7. Future Learning Plans

My next goal is to practise exception handling beyond a single IOException and learn how to validate input before it causes a problem. I also want to use more collection types, such as HashMap, and understand when each collection is suitable. For GUI development, I plan to

learn layout managers so that components can adjust more cleanly than a fixed null layout. These topics will help me build programs that are more reliable and easier to maintain.

As a BCSSE student, I would like to combine the tutorial concepts in a small Student Management System. It could use classes for students and courses, encapsulation for the data, inheritance where it is appropriate, file storage, and a simple interface. I also plan to learn basic unit testing and improve my clean-code habits. Continuing to use GitHub will allow me to record my progress and build a portfolio of coursework and personal practice projects.

8. Conclusion

The ten weeks of tutorials gave me a clear progression from basic Java programs to object-oriented design, file storage, and a Swing interface. The most important improvement was learning how classes can work together instead of treating every program as one long main method. Inheritance and abstraction were challenging, but working through them step by step made the concepts more understandable. I now feel more confident reading, organising, and debugging small Java projects, and I have a clear direction for the skills I want to develop next.

9. GitHub Repository URL

The complete tutorial source code, supporting files, and repository documentation are available at the following GitHub repository:

https://github.com/moha-r/ALI_MOHAMMED_RIYADH_YASEEN_FAREA_202501010268_Object_Oriented_Programming